

Reviewer Pack — Bounded Arithmetic Reflection Strength Lower-Bounds Resoluti...

Entry a88cc518d291 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-30 12:42:54 UTC

Bounded Arithmetic Reflection Strength Lower-Bounds Resolution Refutation Depth

Entry ID: a88cc518d291 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-30 12:42:54 UTC

1. Conjecture

Field A (mathematical branch): Bounded arithmetic (S_2^1 vs T_2^1) **Field B** (complexity object): Resolution refutation depth

Statement:

For any family of unsatisfiable 3-CNF formulas F_n with n variables, the depth of any resolution refutation of F_n is at least the number of iterations of bounded collection ($B\Sigma_1$) required in S_2^1 to prove the consistency of F_n . If T_2^1 proves the consistency of F_n with $O(\log n)$ nested applications of induction, then $\text{depth}(F_n \vdash \perp) = \Omega(n^\varepsilon)$ for some $\varepsilon > 0$.

Rationale (proposer's reasoning):

Bounded arithmetic theories like S_2^1 and T_2^1 classify the strength of feasible reasoning via definable functions and provable consistency statements. The number of times collection or induction is iterated to prove consistency reflects the non-uniform 'logical strength' needed to refute the formula. Resolution depth is sensitive to such proof-theoretic bottlenecks, especially in PHP or Tseitin families where depth-separation is known.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d3651d008504aae6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Resolution refutation depth vs $B\Sigma_1$ iterations

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.8s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3sat(n: int) -> list:
        clauses = []
        for _ in range(2**n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if all(abs(x) != abs(y) for x, y in zip(clause, clause[1:])):
                clauses.append(clause)
        return clauses

    def is_satisfiable(clauses: list) -> bool:
        n = max(abs(var) for var in set(clause for clause in clauses for var in clause))
        assignment = [random.choice([True, False]) for _ in range(n)]
        for clause in clauses:
            if all(not (assignment[abs(var) - 1] ^ (var < 0)) for var in clause):
                return True
        return False

    def resolution_refutation_depth(clauses: list) -> int:
        stack = []
        while not is_satisfiable(clauses):
            unit_clauses = [c for c in clauses if len(c) == 1]
            if not unit_clauses:
                break
            unit_clause = unit_clauses[0]
            var = unit_clause[0]
            new_clause = [-var]
            stack.append((clauses, unit_clause))
            clauses = [c for c in clauses if var not in c and -var not in c]
            for clause in clauses:
                if var in clause:
                    new_clauses = [list(set(c) | set(new_clause)) for c in clauses if -var not in c]
                    clauses.extend(new_clauses)
```

```

        elif -var in clause:
            new_clause = list(set(clause) - {-var})
            stack.append((clauses, unit_clause))
        return len(stack)

def bounded_collection_iterations(n: int) -> int:
    k_n = 0
    while n > 1:
        n = math.ceil(math.log2(n))
        k_n += 1
    return k_n

n = random.randint(5, 40)
clauses = generate_3sat(n)
depth = resolution_refutation_depth(clauses)
k_n = bounded_collection_iterations(n)

c = 0.5 # Empirical constant for the test
if depth < c * k_n:
    return {
        "metric_name": "depth vs  $B\Sigma_1$  iterations",
        "metric_value": depth,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": f"Depth {depth} is less than {c * k_n}"
    }
else:
    return {
        "metric_name": "depth vs  $B\Sigma_1$  iterations",
        "metric_value": depth,
        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 7 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_depth = sum(r["metric_value"] for r in results) / len(results)
    std_depth = math.sqrt(sum((r["metric_value"] - mean_depth)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_depth} std={std_depth} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='depth < c * k_n' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data to determine support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out without producing conclusive data; support condition cannot be evaluated.
| next: Increase timeout duration and re-run test with stricter parameter monitoring

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	cerebras	qwen-3-235b-a22b-ins	0	0	11647
2	preregistration	groq	llama-3.3-70b-versat	0	0	4552
3	novelty	ollama_remote	qwen3:8b	0	0	22461
4	novelty	ollama_remote	qwen3:8b	0	0	14504
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15940
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10524
7	judge	ollama_remote	qwen3:8b	0	0	98050

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 177679 ms total latency. Provider mix: {'cerebras': 1, 'groq': 1, 'ollama_remote': 5}

(full prompt+response transcripts available in `research/audit/a88cc518d291.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a88cc518d291.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/a88cc518d291.tar.gz` (if generated)